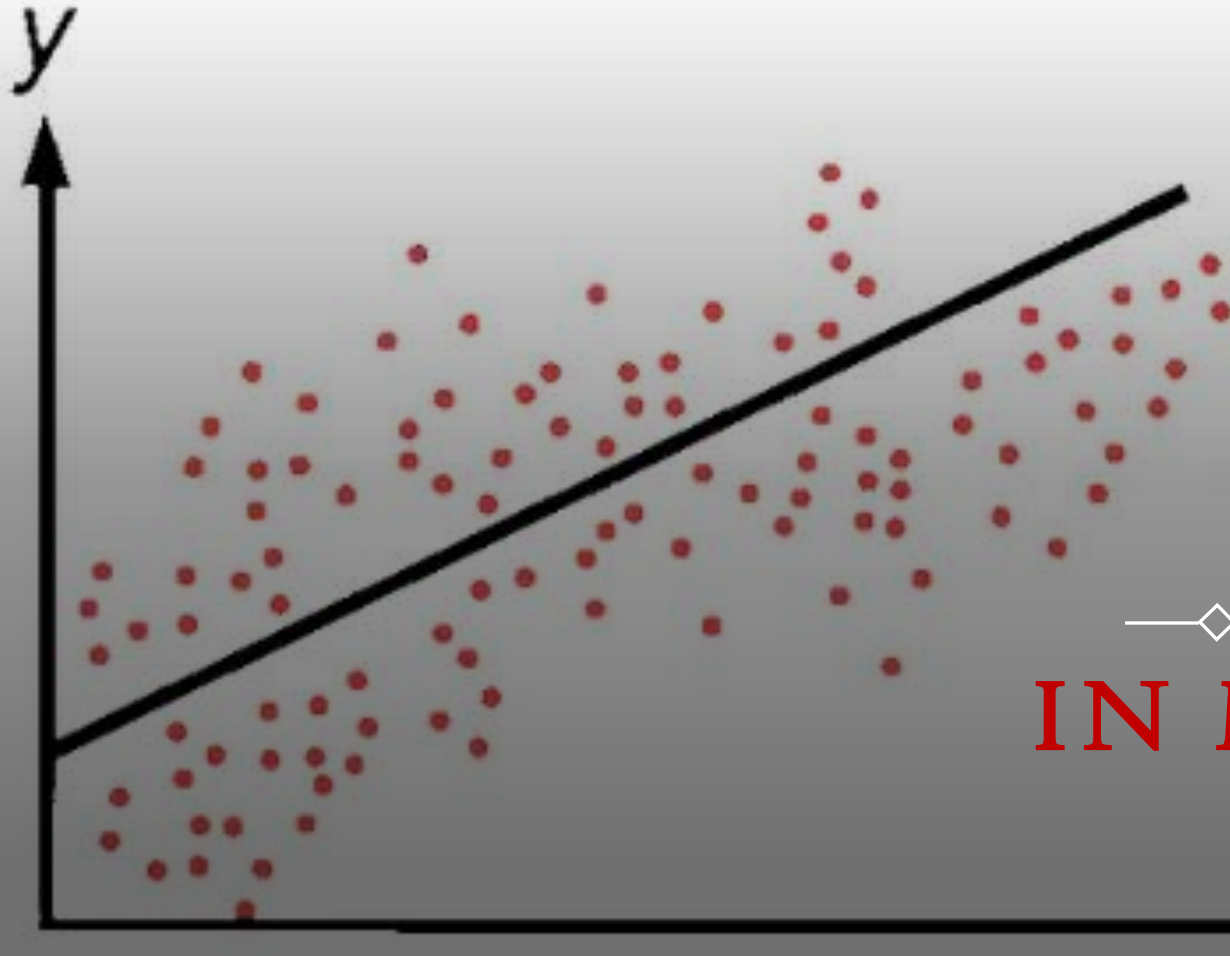


Linear Regression



—◇—
IN MACHINE LEARNING

By. Abhijeet Palai

❑ Explanation of Linear Regression Concept with the help of example:

1. CREATE DATASET

```
[7]:  
# Create dataset as dictionary  
data_dict = {  
    'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10,  
                     11, 12, 13, 14, 15, 16, 17, 18, 19, 20],  
    'Exam_Score': [25, 30, 35, 45, 50, 55, 60, 65, 70, 75,  
                  78, 82, 85, 87, 89, 91, 92, 93, 94, 95]  
}
```

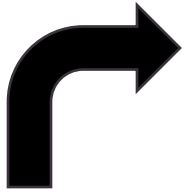


Dataset:

	Hours_Studied	Exam_Score
0	1	25
1	2	30
2	3	35
3	4	45
4	5	50

Dataset shape: (20, 2)

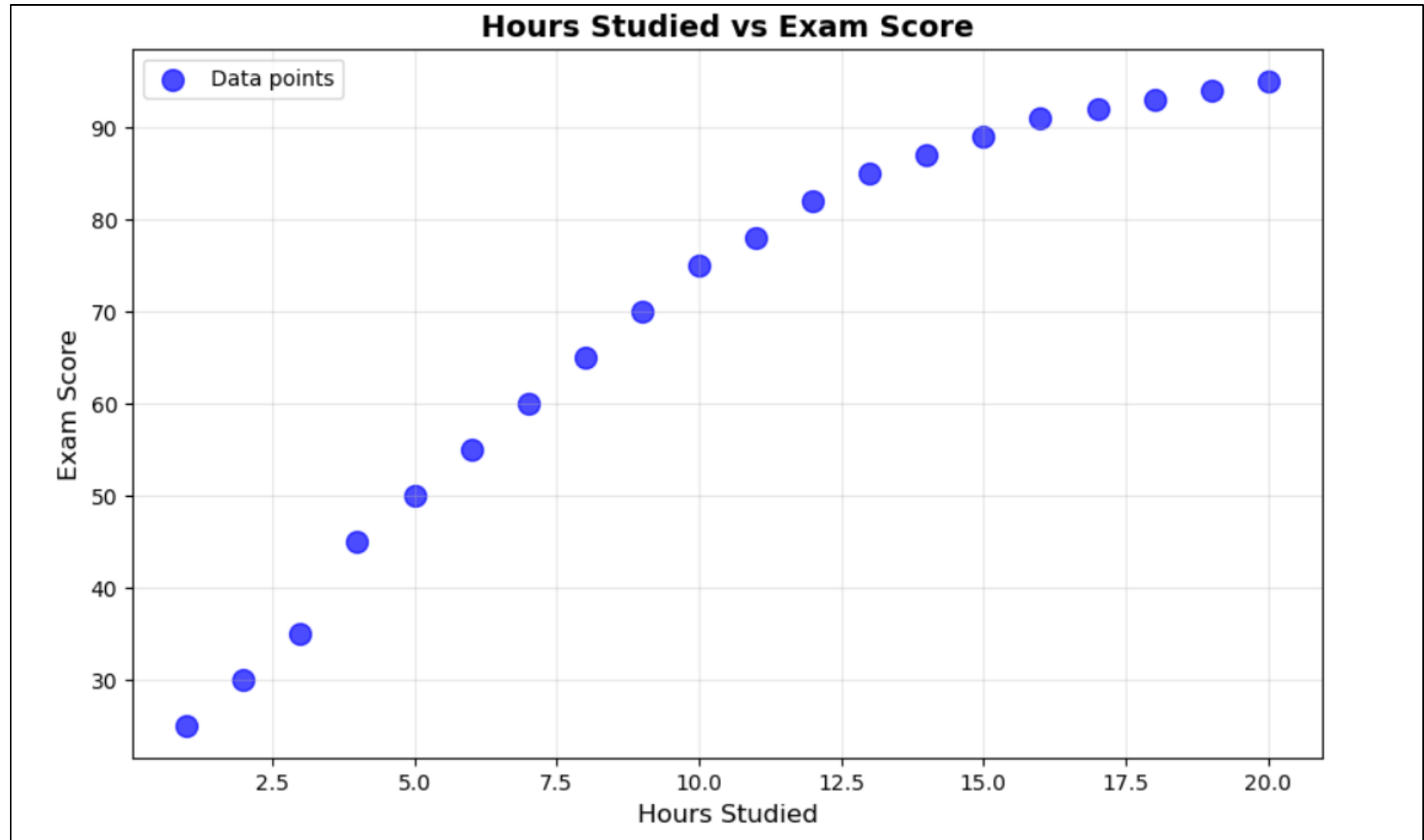
2. VISUALISE THE DATASET



Dataset:

	Hours_Studied	Exam_Score
0	1	25
1	2	30
2	3	35
3	4	45
4	5	50

Dataset shape: (20, 2)



3. COST (LOSS) FUNCTIONS IN REGRESSION

Cost functions measure **how wrong a model's predictions are**.

➤ VARIOUS COST FUNCTIONS

a) Mean Squared Error (MSE)

- Penalizes large errors more
- Smooth and differentiable
- Sensitive to outliers

c) Root Mean Squared Error (RMSE)

- Square root of MSE
- Same unit as target variable
- Still sensitive to outliers

b) Mean Absolute Error (MAE)

- Treats all errors linearly
- Robust to outliers
- Not differentiable at zero

d) Huber Loss

- Combines MSE (small errors) + MAE (large errors)
- Balanced and robust loss function
- Requires choosing a threshold (δ)

Purpose: Compare how different losses behave for the same prediction errors .

➤ Testing different “fake models”

manually creating 3 different prediction lines and checking which gives less loss.

$y_{\text{pred_simple}} = 4 \cdot X + 20$
 $y_{\text{pred_better}} = 3.5 \cdot X + 25$
 $y_{\text{pred_bad}} = 2 \cdot X + 10$

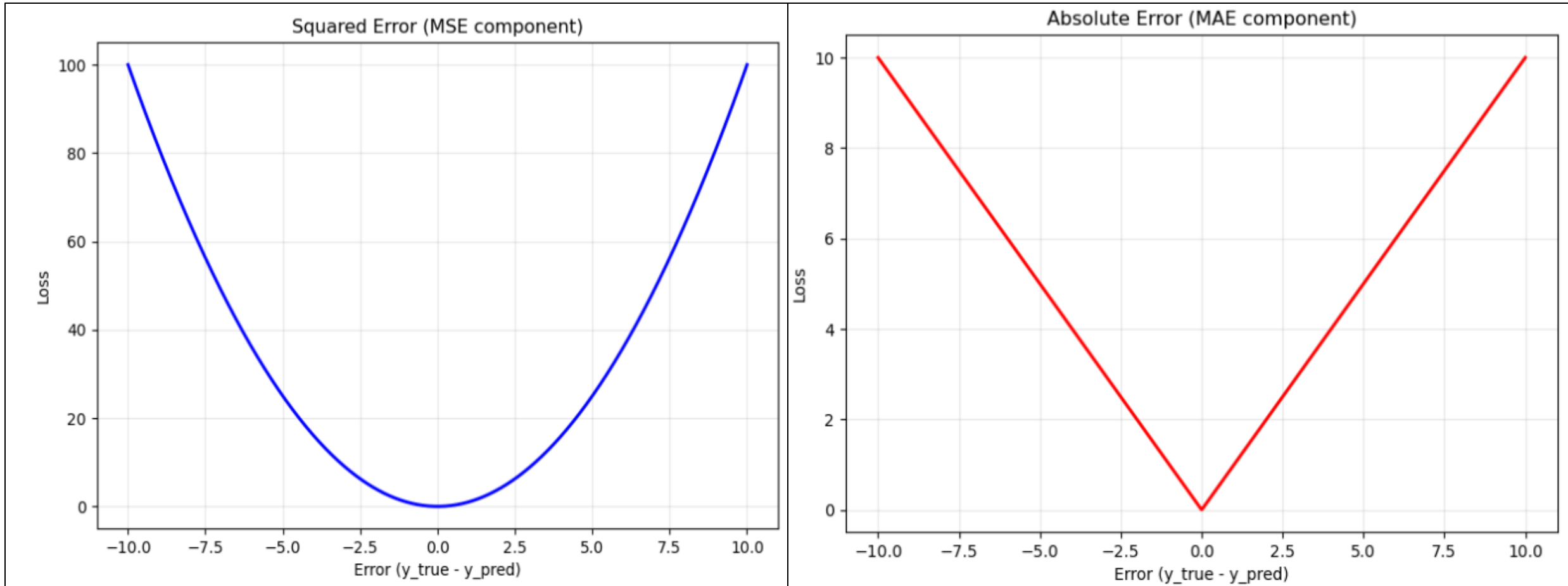


2. COST FUNCTIONS COMPARISON

Cost Functions Comparison for Different Models:

Model	MSE	MAE	RMSE	Huber
Simple Model ($4x + 20$)	95.400	8.50	9.767292	8.00000
Better Model ($3.5x + 25$)	99.275	8.65	9.963684	8.18125
Bad Model ($2x + 10$)	1639.400	38.80	40.489505	38.30000

➤ Plot of Different Loss Function

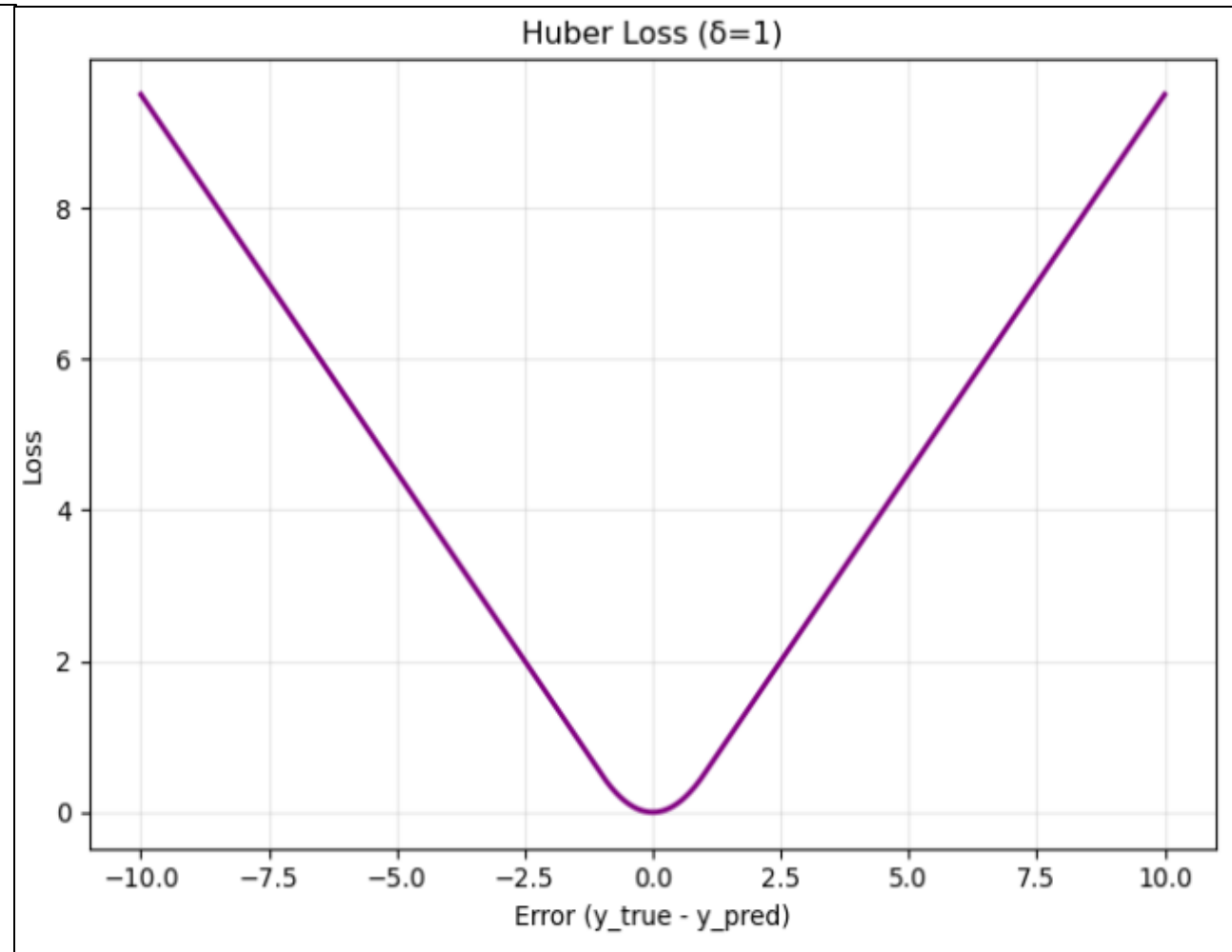
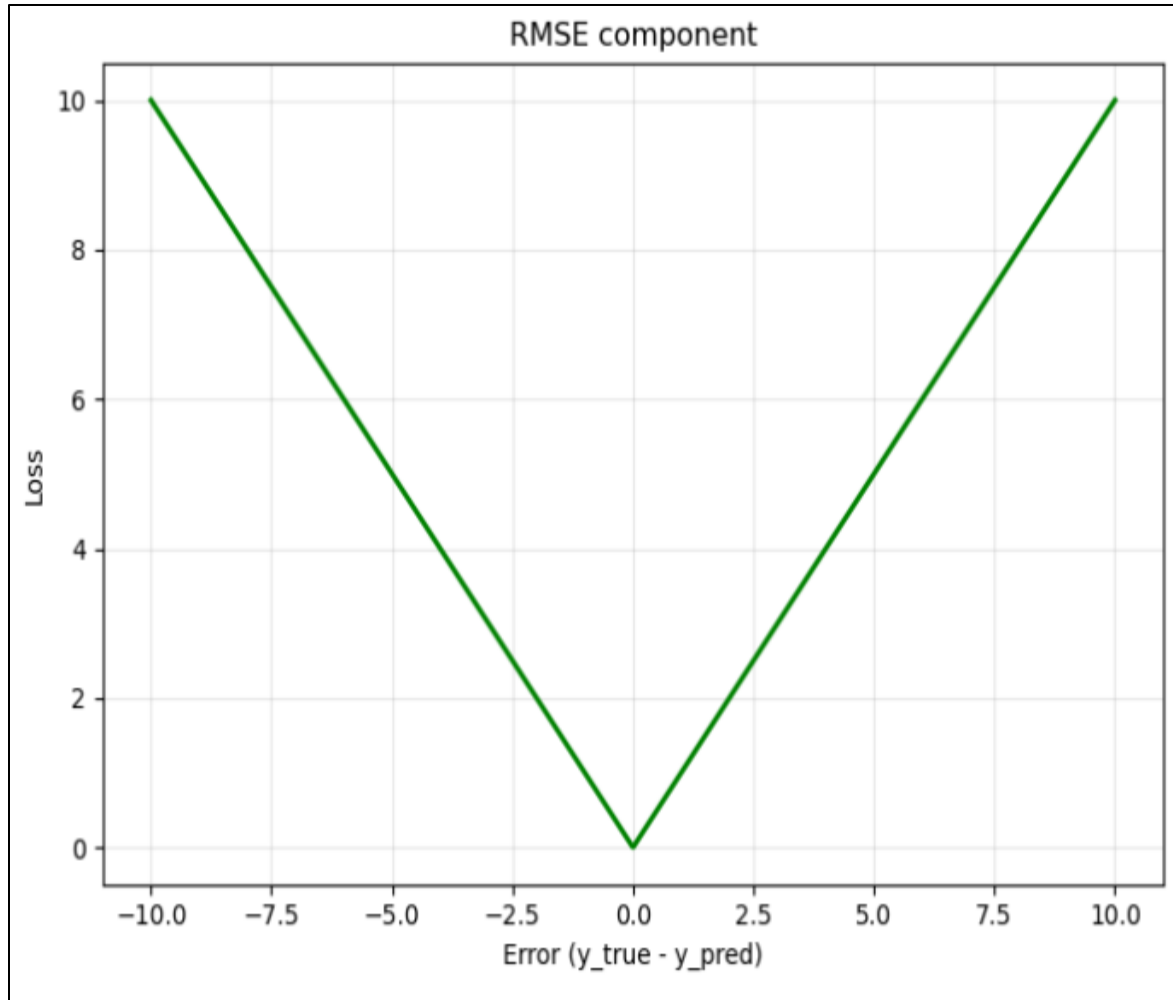


$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

y_i : true value
 $\hat{y}_i$: predicted value
 n : number of data points

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$r = y_i - \hat{y}_i$$



$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

$$L_{\delta}(r) = \begin{cases} \frac{1}{2}r^2, & \text{if } |r| \leq \delta \\ \delta (|r| - \frac{1}{2}\delta), & \text{if } |r| > \delta \end{cases}$$

$$\text{Huber Loss} = \frac{1}{n} \sum_{i=1}^n L_{\delta}(y_i - \hat{y}_i)$$

➤ Which Cost Function is Best?

1.MSE (Mean Squared Error):

- Advantages: Differentiable everywhere, emphasizes larger errors
- Disadvantages: Sensitive to outliers

2.MAE (Mean Absolute Error):

- Advantages: Robust to outliers
- Disadvantages: Not differentiable at 0

3.RMSE (Root Mean Squared Error):

- Advantages: Same units as target variable
- Disadvantages: Same sensitivity to outliers as MSE

4.Huber Loss:

- Advantages: Combines benefits of MSE and MAE
- Disadvantages: Requires tuning of delta parameter

RECOMMENDATION: For this dataset without outliers, MSE/RMSE are good choices. If you expect outliers, consider MAE or Huber Loss.

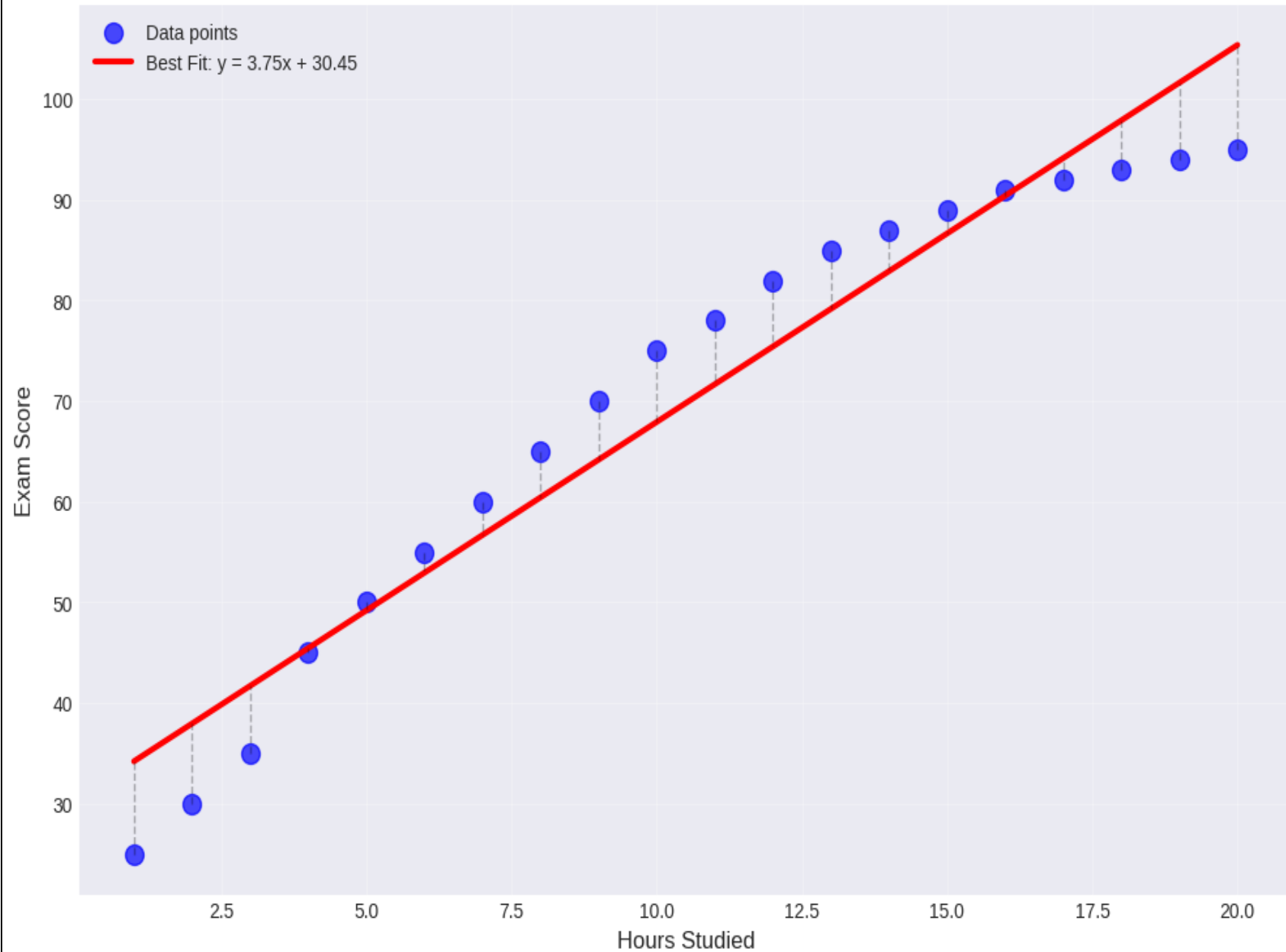
4. FITTING BEST LINE USING LEAST SQUARES

➤ Best-Fit Line (Least Squares Method)

Used to find the optimal slope and intercept mathematically

- **Simple Linear Regression**
 - Relationship modeled as a straight line
- **Least Squares Principle**
 - Minimizes total squared residuals
- **Residuals**
 - Vertical distance between actual and predicted values
- **Analytical (Closed-Form) Solution**
 - Exact solution when data size is small

Best Fit Line with Residuals



Best Fit Line Parameters:

Slope (m): 3.7474

Intercept (b): 30.4526

Equation: $y = 3.7474 * x + 30.4526$

MSE on training data: 32.4379

RMSE on training data: 5.6954

$$\hat{y} = mx + b$$

$$m = \frac{\sum(x - \bar{x})(y - \bar{y})}{\sum(x - \bar{x})^2}$$

$$b = \bar{y} - m\bar{x}$$

5. GRADIENT DESCENT OPTIMIZATION/CONVERGENCE

Scalable method for large datasets and complex models

- Iterative Optimization Algorithm
- Used when analytical solution is expensive or impossible
- Updates parameters step-by-step to reduce loss

Key Concepts

- **Learning Rate**
 - Controls step size
- **Convergence**
 - Cost decreases over iterations
- **Cost Surface**
 - 3D view of loss vs parameters
- **Optimization Path**
 - How parameters move toward minimum

Training with Gradient Descent:

Iteration 0:	Cost = 2837.7998,	Slope = 1.7150,	Intercept = 0.1396
Iteration 100:	Cost = 222.6441,	Slope = 5.8472,	Intercept = 1.8016
Iteration 200:	Cost = 205.8895,	Slope = 5.7525,	Intercept = 3.0925
Iteration 300:	Cost = 190.6108,	Slope = 5.6622,	Intercept = 4.3253
Iteration 400:	Cost = 176.6780,	Slope = 5.5759,	Intercept = 5.5026
Iteration 500:	Cost = 163.9724,	Slope = 5.4935,	Intercept = 6.6268
Iteration 600:	Cost = 152.3860,	Slope = 5.4148,	Intercept = 7.7003
Iteration 700:	Cost = 141.8203,	Slope = 5.3397,	Intercept = 8.7255
Iteration 800:	Cost = 132.1852,	Slope = 5.2680,	Intercept = 9.7045
Iteration 900:	Cost = 123.3988,	Slope = 5.1995,	Intercept = 10.6394

Final Parameters (Gradient Descent):

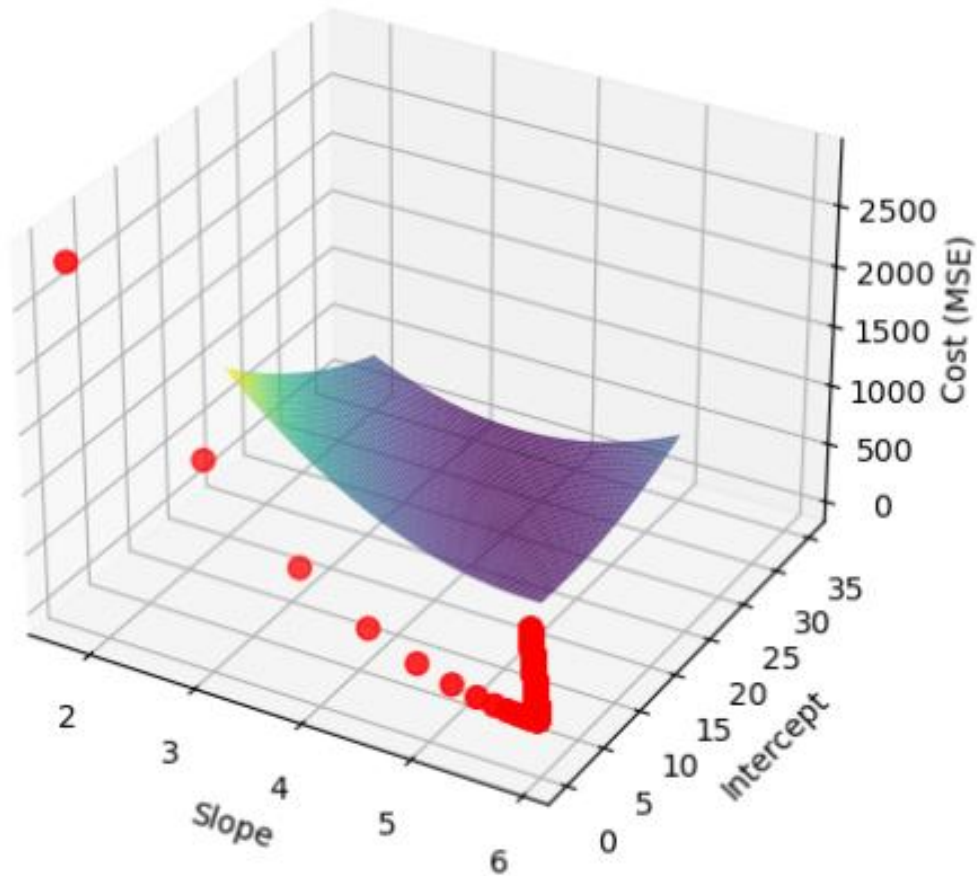
Slope: 5.1347

Intercept: 11.5234

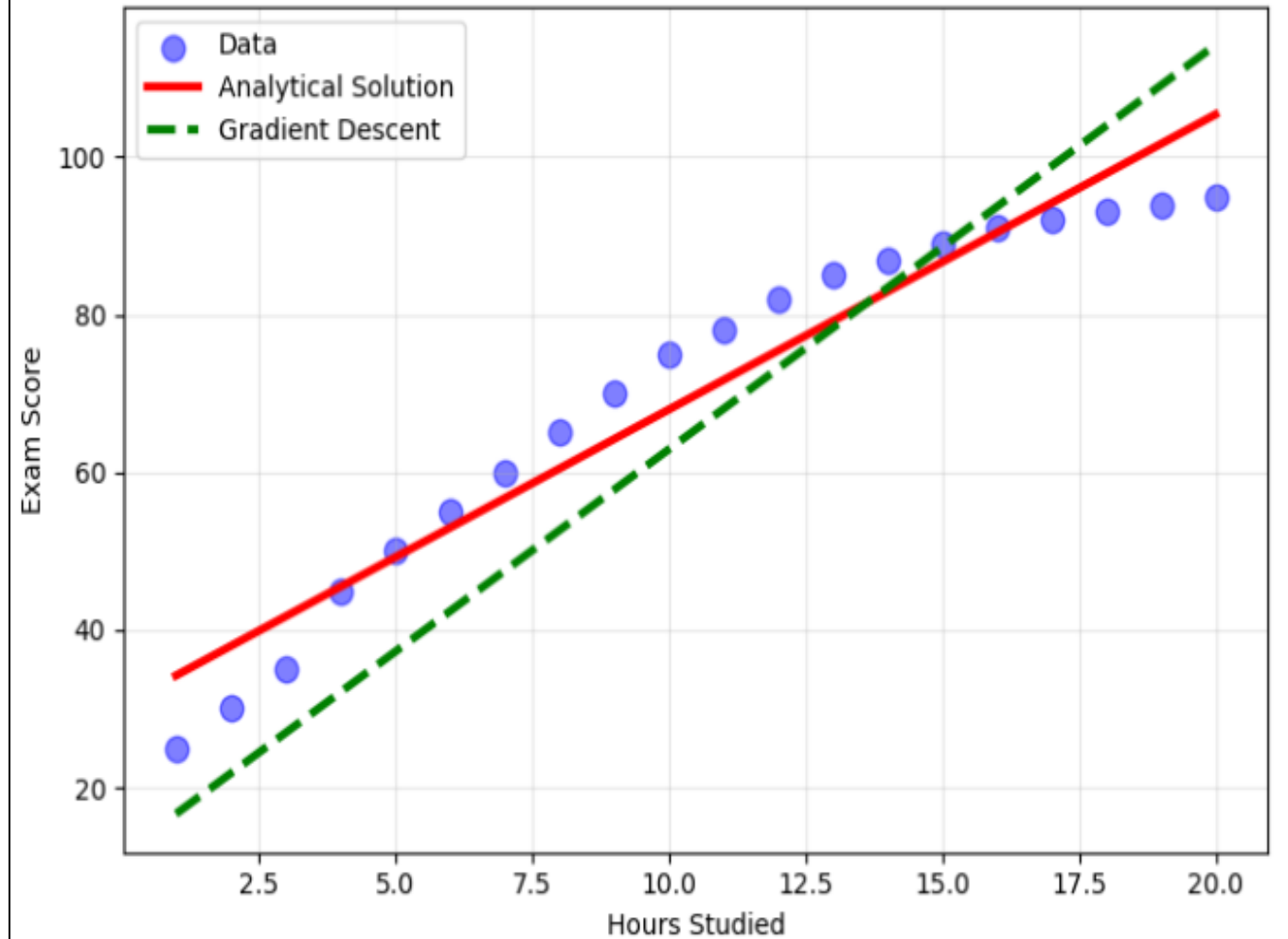
Final MSE: 115.4629

➤ Comparison: Analytical Solution vs Gradient Descent

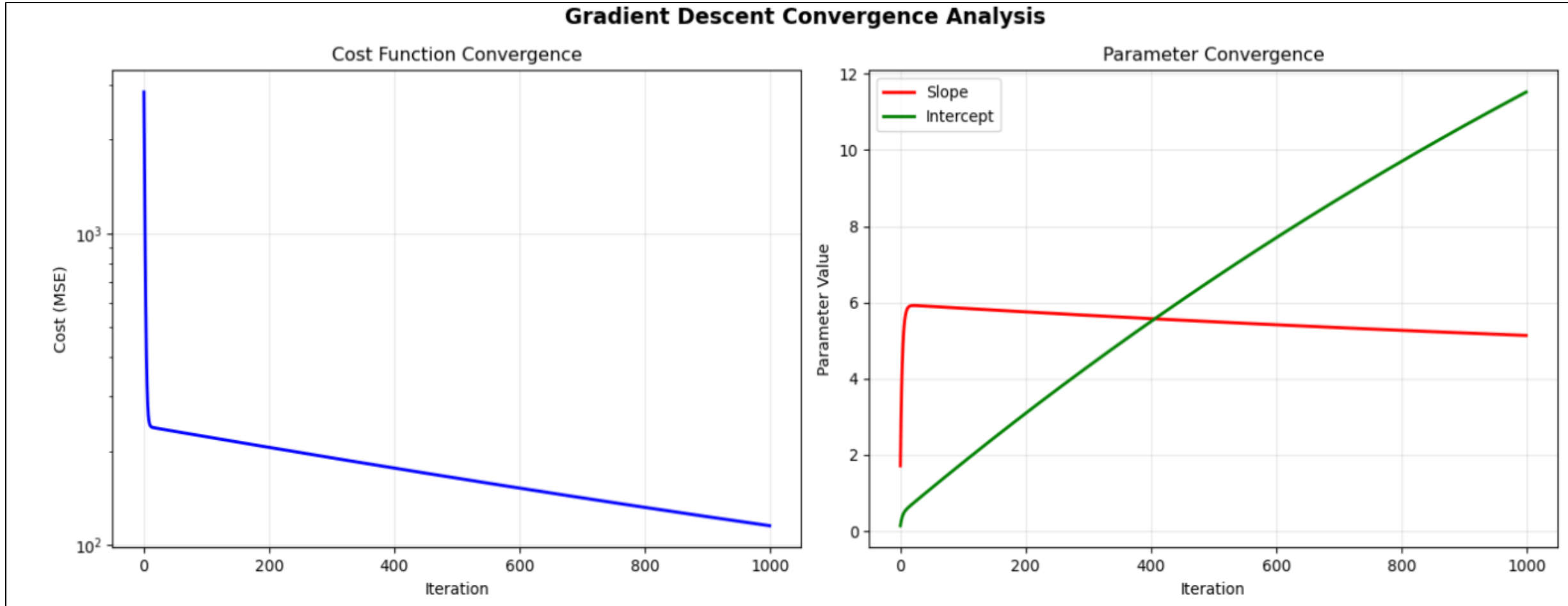
Cost Surface and Gradient Descent Path



Comparison: Analytical vs Gradient Descent



➤ Gradient Descent Convergence Analysis



6. REGULARIZATION (Overfitting Control)

Regularization adds a **penalty** to prevent overly complex models.

a) Ridge Regression (L2)

- Penalizes squared magnitude of coefficients
- Shrinks parameters smoothly
- Handles multicollinearity well

b) Lasso Regression (L1)

- Penalizes absolute magnitude
- Can force coefficients to zero
- Performs feature selection

c) Elastic Net

- Combination of L1 + L2
- More stable when features are correlated

Prevents overfitting and improves generalization

$$\beta = (X^T X + \lambda I)^{-1} X^T y$$

$$J_{\text{Ridge}}(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

$$J_{\text{Lasso}}(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

Applying Different Regularization Techniques:

Comparison of Regularization Techniques:

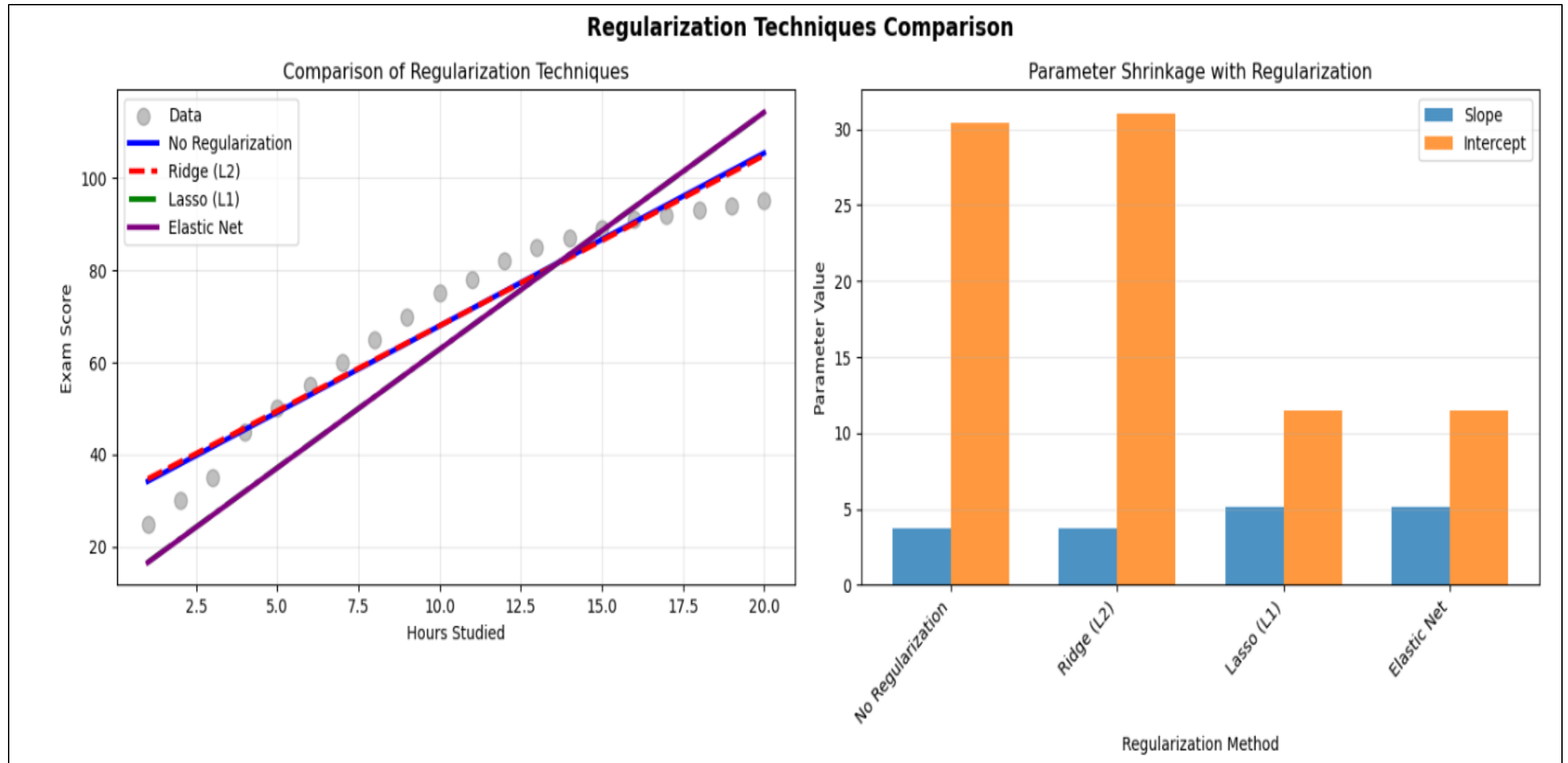
	Method	Slope	Intercept	MSE	L1 Norm	L2 Norm
No Regularization		3.747368	30.452632	32.437895	34.200000	941.405540
Ridge (L2)		3.691852	31.035556	32.540374	34.727407	976.835479
Lasso (L1)		5.135922	11.504947	115.624892	16.640869	158.741503
Elastic Net		5.140325	11.441509	116.182716	16.581834	157.331070

$\lambda \geq 0$: overall regularization strength

$\alpha \in [0, 1]$: mix ($\alpha = 1 \rightarrow \text{Lasso}$, $\alpha = 0 \rightarrow \text{Ridge}$)

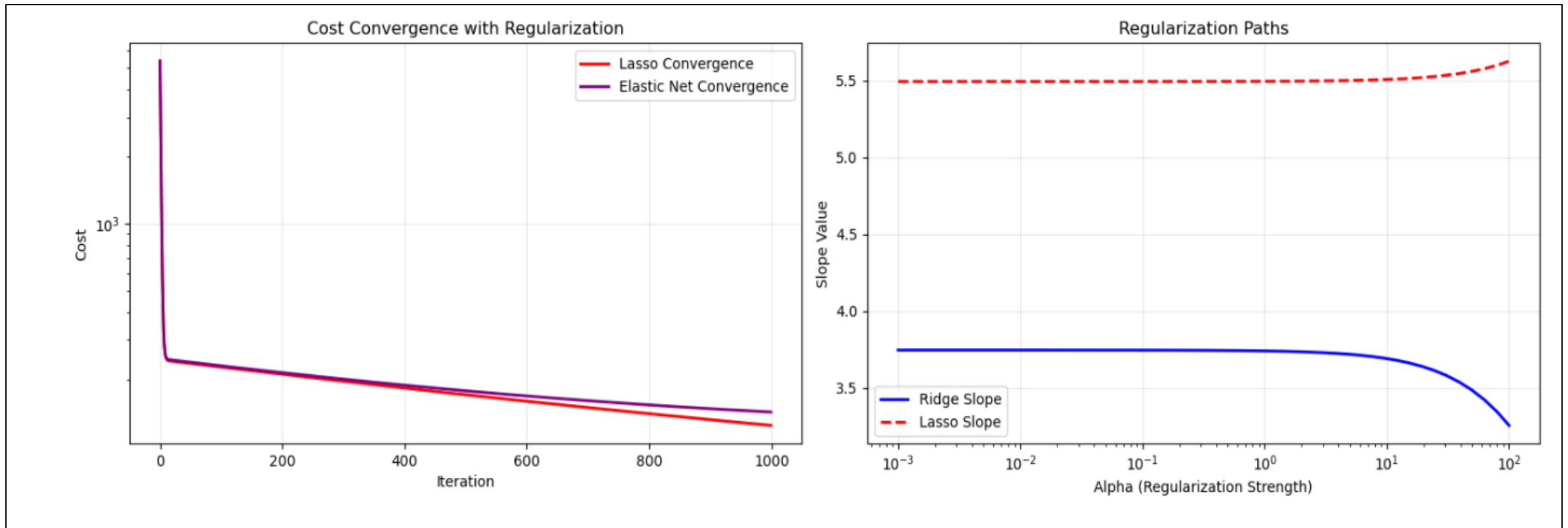
$$J_{\text{ElasticNet}}(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left[\alpha \sum_{j=1}^p |\beta_j| + (1 - \alpha) \sum_{j=1}^p \beta_j^2 \right]$$

➤ Regularization Techniques Comparison



➤ Regularization Path

- Shows how model parameters change with regularization strength (α)
- Helps understand:
 - Bias–variance tradeoff
 - Coefficient shrinkage behavior



➤ Summary of Regularization Techniques

1. Ridge Regression (L2):

- Adds squared magnitude of coefficients to cost
- Shrinks coefficients but never to exactly zero
- Good for multicollinearity

2. Lasso Regression (L1):

- Adds absolute magnitude of coefficients to cost
- Can shrink coefficients to exactly zero (feature selection)
- Good for sparse models

3. Elastic Net:

- Combines L1 and L2 regularization
- Good when features are correlated
- More stable than Lasso

For this simple linear regression:

- All methods give similar results since we only have one feature
- In high-dimensional problems, regularization helps prevent overfitting
- Choice depends on whether feature selection is needed (Lasso) or just shrinkage (Ridge)

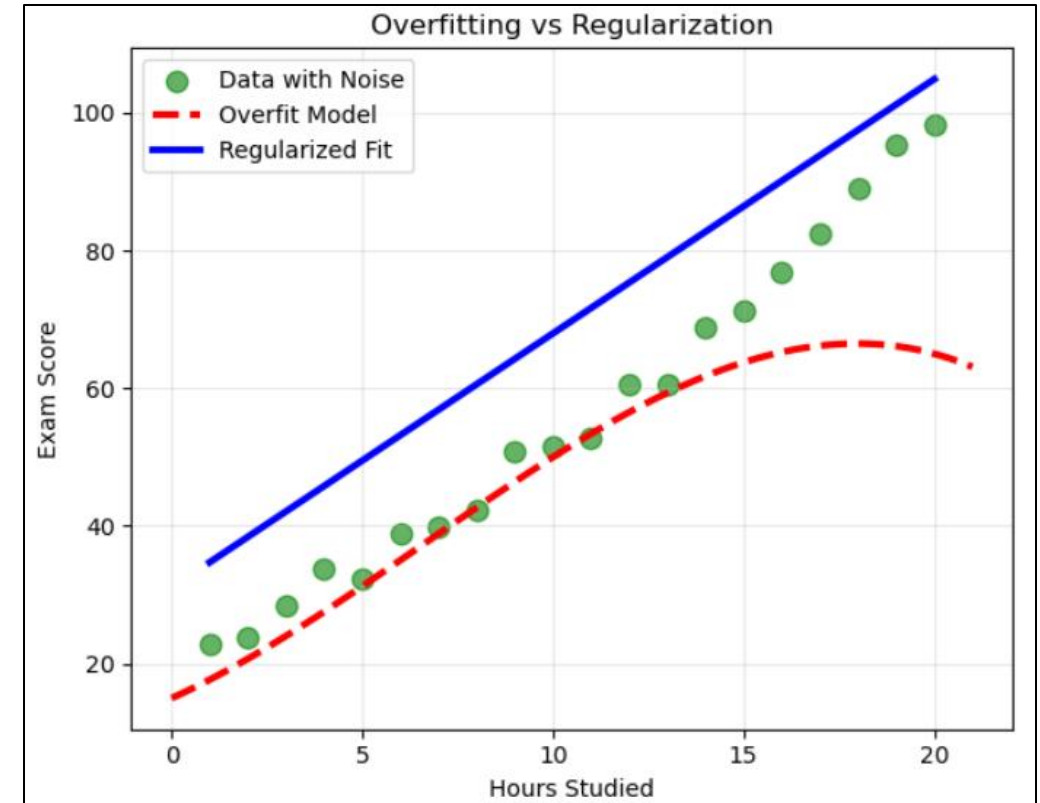
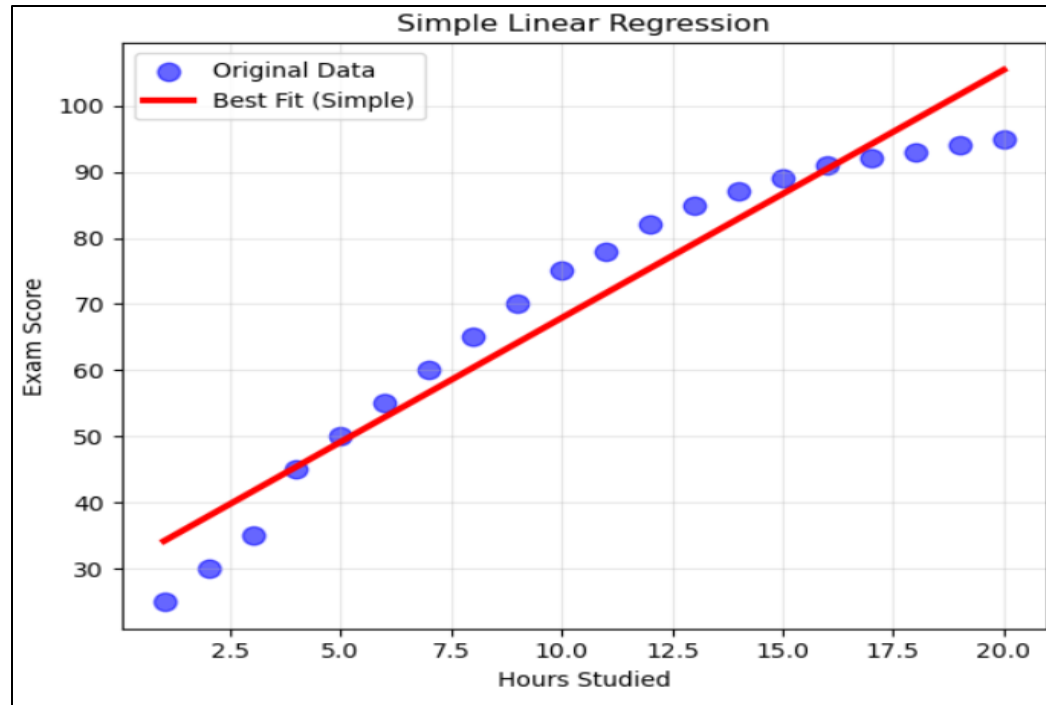
7. OVERFITTING VS REGULARIZATION

Overfitting

- Model captures noise instead of pattern
- Common in high-degree polynomial models

Regularization

- Controls complexity
- Produces smoother, more generalizable models



Key idea: simpler models often generalize better

➤ Comparison: Analytical Solution vs Gradient Descent

- **Analytical Method**
 - Fast, exact, limited to simple models
- **Gradient Descent**
 - Approximate, slower, but scalable
 - Both aim to minimize the same loss function

Different roads to the same optimum

➤ KEY TAKEAWAYS

1. COST FUNCTIONS:

- MSE: Best for normally distributed errors
- MAE: Robust to outliers
- RMSE: Interpretable in original units
- Huber: Best of both worlds

2. BEST FIT LINE:

- Analytical solution via normal equations is exact
- Gradient descent is iterative but scalable

3. CONVERGENCE:

- Gradient descent finds minimum iteratively
- Learning rate is crucial for convergence
- Batch gradient descent shown here

4. REGULARIZATION:

- Prevents overfitting
- Ridge (L2): Shrinks coefficients
- Lasso (L1): Performs feature selection
- Elastic Net: Combines both

For simple linear regression with one feature:

- Analytical solution is preferred
- Regularization has limited effect
- Choice depends on data characteristics

8. Model Evaluation & Comparison

Comparing:

- Loss values (MSE, RMSE)
- Parameter magnitudes (L1/L2 norms)

Understanding tradeoffs between:

- Accuracy
- Stability
- Interpretability

9. Core Machine Learning Takeaways

- Loss functions define *what “best” means*
- Optimization finds *how to reach best*
- Regularization controls *model complexity*
- Visualization helps *intuition and debugging*